Arquitetura do Data Lake Local — NLST JOIN Optimization

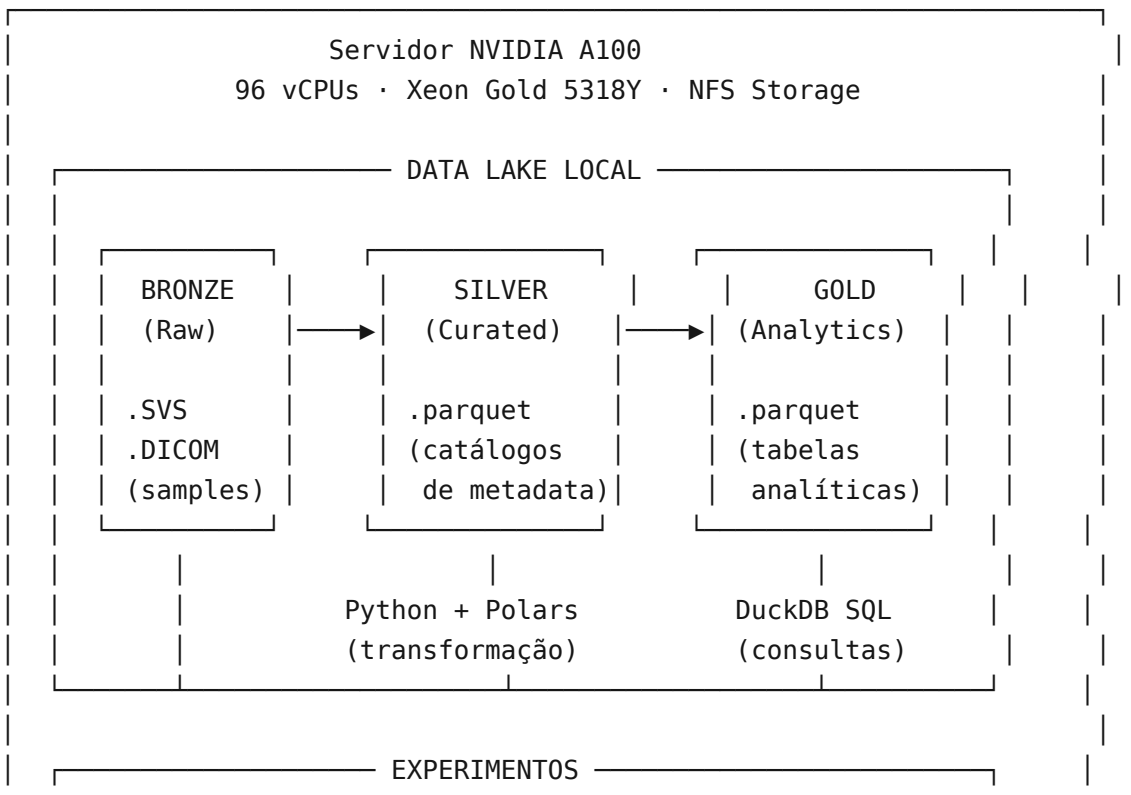
Visão Geral

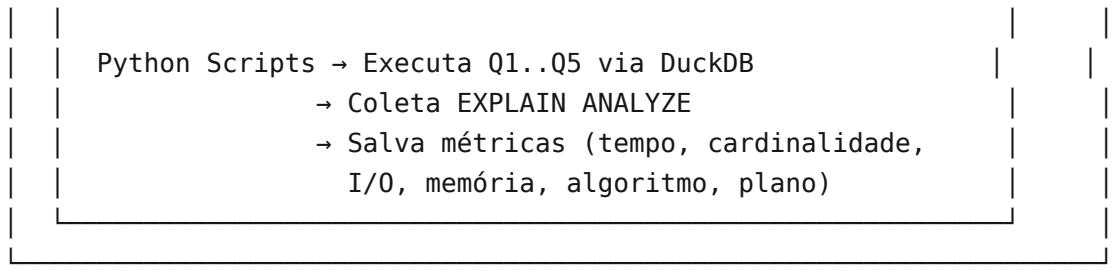
Data lake local no servidor NVIDIA A100 (96 vCPUs, Xeon Gold 5318Y) para suportar experimentos de otimização de JOINs sobre dados do NLST. Stack 100% gratuita e open-source.

Componente	Ferramenta	Papel
Armazenamento	Filesystem NFS + formatos nativos (Bronze) e Parquet (Silver/Gold)	Data lake local com arquitetura medalhão
Transformação	Python + Polars	Extração de metadados dos arquivos médicos → Parquet
Motor SQL	DuckDB	Consultas analíticas, EXPLAIN ANALYZE, experimentos de JOIN
Coleta de métricas	Python scripts	Automatização dos experimentos e coleta de resultados

---

Diagrama da Arquitetura





Estrutura de Diretórios

```
/nfs/home/lucas_piovani/
├── NLST-Datasets/                          # Dados originais do NLST (já existente)
├── NLST-Catalogs/                          # Catálogos já construídos (já existente)
├── PUB-NLST-*/                             # Notebooks de exploração (já existentes)
│
└── nlst-datalake/                          # ← NOVO: Data Lake Local
    │
    ├── bronze/                             # Camada Raw — dados brutos, sem transformação
    │   ├── svs_samples/                    # Amostras de arquivos .SVS (patologia)
    │   └── dicom_samples/                  # Amostras de arquivos .DICOM (imagens)
    │
    ├── silver/                             # Camada Curated — metadados extraídos, limpos
    │   ├── cancer_metadata.parquet         # Catálogo de metadados de câncer
    │   ├── pathology_metadata.parquet      # Catálogo de metadados de patologia
    │   └── participant_metadata.parquet    # Catálogo de metadados de participantes
    │
    ├── gold/                               # Camada Analytics — tabelas prontas para
consulta
    │   ├── cancer.parquet                  # Tabela analítica de câncer
    │   ├── pathology.parquet               # Tabela analítica de patologia
    │   └── participant.parquet             # Tabela analítica de participantes
    │
    ├── experiments/                         # Resultados dos experimentos
    │   ├── queries/                         # Q1.sql .. Q5.sql
    │   ├── plans/                           # Planos de execução salvos (JSON/TXT)
    │   └── results/                         # Métricas coletadas (CSV)
    │
    ├── scripts/                             # Scripts de pipeline e experimentos
    │   ├── 01_ingest_bronze.py             # Copia samples para bronze/
    │   ├── 02_extract_silver.py            # Extrai metadados → Parquet (Polars)
    │   ├── 03_build_gold.py                # Silver → Gold: joins, limpeza final (Polars)
    │   ├── 04_run_experiments.py           # Executa queries no DuckDB, coleta métricas
    │   └── 05_analyze_results.py           # Análise e visualização dos resultados
    │
    └── nlst.duckdb                          # Banco DuckDB (criado pelo script 04)
```

Fluxo de Dados por Camada

Bronze (Raw)

- **O que entra:** amostras dos arquivos originais .SVS e .DICOM do NLST
- **Formato:** formatos nativos (binários médicos)
- **Transformação:** nenhuma — cópia fiel dos originais
- **Propósito:** preservar a linhagem dos dados e demonstrar a origem no artigo

Silver (Curated)

- **O que entra:** catálogos de metadados extraídos dos arquivos médicos
- **Formato:** Parquet
- **Transformação:** Python + Polars — extração de metadados, tipagem, limpeza, deduplicação
- **Propósito:** dados estruturados e limpos, prontos para serem combinados
- **Exemplos de metadados:**
  - De .DICOM: patient_id, study_date, modality, series_description, pixel_spacing, slice_thickness...
  - De .SVS: slide_id, patient_id, magnification, image_dimensions, staining_type...
  - De datasets clínicos: patient_id, age, gender, smoking_status, cancer_type, stage...

Gold (Analytics)

- **O que entra:** tabelas Silver já integradas e enriquecidas
- **Formato:** Parquet
- **Transformação:** Polars — joins entre catálogos, criação de features analíticas
- **Propósito:** tabelas finais sobre as quais o DuckDB executará as consultas experimentais

---

Motor de Consulta: DuckDB

O DuckDB atua como SGBD analítico sobre os Parquets da camada Gold.

Por que DuckDB é ideal para este artigo

1. **EXPLAIN ANALYZE detalhado** — mostra tempos reais por operador, cardinalidades, I/O
2. **Leitura nativa de Parquet** — sem necessidade de importação, lê direto do filesystem
3. **Motor colunar vetorizado** — performance excelente para workloads analíticos
4. **Controle de algoritmos de JOIN** — pragmas para forçar estratégias específicas
5. **Zero configuração** — `pip install duckdb`, sem servidor, sem daemon
6. **Reprodutibilidade** — banco em arquivo único (`nlst.duckdb`), fácil de versionar

Exemplo de uso nos experimentos

```
-- Q1: JOIN básico entre Cancer e Pathology
EXPLAIN ANALYZE
SELECT c.*, p.*
FROM cancer c
JOIN pathology p ON c.pid = p.pid;

-- Q2: JOIN com filtro seletivo (pushdown)
EXPLAIN ANALYZE
SELECT c.*, p.*
FROM cancer c
JOIN pathology p ON c.pid = p.pid
```

```
WHERE c.de_type = 1;

-- Q4: JOIN de 3 tabelas — ordem importa
EXPLAIN ANALYZE
SELECT c.*, p.*, pt.*
FROM cancer c
JOIN pathology p ON c.pid = p.pid
JOIN participant pt ON c.pid = pt.pid;
```

Coleta de Métricas (Python Scripts)

O script `04_run_experiments.py` automatiza a execução e coleta:

```
import duckdb
import time
import json
import polars as pl

con = duckdb.connect("nlst-datalake/nlst.duckdb")

# Registrar Parquets como tabelas
con.execute("CREATE TABLE cancer AS SELECT * FROM read_parquet('nlst-
datalake/gold/cancer.parquet')")
con.execute("CREATE TABLE pathology AS SELECT * FROM read_parquet('nlst-
datalake/gold/pathology.parquet')")
con.execute("CREATE TABLE participant AS SELECT * FROM read_parquet('nlst-
datalake/gold/participant.parquet')")

# Definir queries
queries = {
    "Q1": "SELECT c.*, p.* FROM cancer c JOIN pathology p ON c.pid = p.pid",
    "Q2": "SELECT c.*, p.* FROM cancer c JOIN pathology p ON c.pid = p.pid WHERE c.de_type
= 1",
    # ... Q3, Q4, Q5
}

results = []
for name, sql in queries.items():
    # Coletar plano de execução
    plan = con.execute(f"EXPLAIN ANALYZE {sql}").fetchall()

    # Medir tempo de execução (múltiplas rodadas)
    times = []
    for _ in range(10):
        start = time.perf_counter()
        con.execute(sql).fetchall()
        elapsed = time.perf_counter() - start
        times.append(elapsed)
```

```
    results.append({
        "query": name,
        "mean_time_s": sum(times) / len(times),
        "min_time_s": min(times),
        "max_time_s": max(times),
        "plan": str(plan),
    })

# Salvar resultados
pl.DataFrame(results).write_parquet("nlst-datalake/experiments/results/metrics.parquet")
```

Validação da Arquitetura vs. Requisitos do Artigo

Requisito do Artigo	Suportado?	Como
Comparar algoritmos de JOIN (Hash, Nested Loop, Merge)	✅	DuckDB EXPLAIN ANALYZE + pragmas
Medir efeito de filtros pré-JOIN	✅	Comparar Q1 vs Q2/Q3
Analisar ordem de JOINs com 3+ tabelas	✅	Q4 com variações de ordem
Coletar tempo de execução	✅	Python `time.perf_counter()`
Coletar cardinalidade intermediária	✅	EXPLAIN ANALYZE mostra por operador
Coletar I/O e memória	✅	EXPLAIN ANALYZE + profiling do DuckDB
Analisar plano do otimizador	✅	EXPLAIN mostra plano estimado
Comparar estimado vs. real	✅	EXPLAIN vs. EXPLAIN ANALYZE
Variar tamanho dos dados	✅	Subconjuntos dos Parquets
Ferramentas 100% gratuitas	✅	Python, Polars, DuckDB — tudo open-source
Roda no servidor local	✅	Sem dependência de cloud ou serviço externo

---

Dependências (requirements.txt)

```
duckdb>=1.1.0
polars>=1.0.0
pydicom>=2.4.0          # Leitura de metadados DICOM
openslide-python>=1.3.1 # Leitura de metadados SVS
matplotlib>=3.8.0       # Visualizações para o artigo
seaborn>=0.13.0         # Visualizações estatísticas
```

Próximos Passos

1. **Criar a estrutura de diretórios** (`mkdir -p nlst-datalake/{bronze,silver,gold,experiments,scripts}`)
2. **Copiar samples para Bronze** (selecionar amostras representativas de .SVS e .DICOM)
3. **Implementar extração de metadados** (script 02 — Polars + pydicom + openslide)
4. **Gerar tabelas Gold** (script 03 — limpeza final e padronização de chaves)
5. **Configurar DuckDB e rodar experimentos** (scripts 04 e 05)

Report generated by Databricks Genie